Description Projet Big Data / MLOps (API GitHub)

Prédiction de résolution d'issues

Sujet

L'objectif serait de collecter les données de dépôts Git et de prédire, quand une issue est ouverte, si et en combien de temps une issue sera résolue en fonction des autres issues déjà réalisées au préalable (probabilité). L'utilité serait d'aider les mainteneurs à prioriser et les contributeurs à choisir où s'investir.

Utilisateurs cibles

- Mainteneurs de projets (tri et priorisation du backlog).
- Entreprises.
- Nouveaux contributeurs qui cherchent des issues avec une chance d'aboutir.

Prédictions

- **Cible principale (classification) :** l'issue est-elle résolue en un certain nombre de jours / semaines ?
- **Cible secondaire (régression) :** le temps avant fermeture (échelle plus précise).
- Le champ *state_reason* de GitHub (*completed* / *not_planned*) permet de distinguer une vraie résolution d'une fermeture sans suite.

Données

Source	Contenu utile
API GitHub : issues	titre, corps, labels, dates, auteur, *author_association*, état
Commentaires et événements	délai de première réponse, nombre de commentaires, mises à jour liées (Pull request)
Métadonnées du dépôt	taille de la communauté, activité récente, langage
GitHub Archive (enrichissement)	historique d'événements pour constituer le jeu d'entraînement

Features

- **Texte :** embeddings du titre et du corps, longueur, présence de code.
- **Contexte :** labels, *author_association*, historique de l'auteur.
- **Dépôt :** délai médian de fermeture des issues, nombre de mainteneurs actifs, nombre d'issues ouvertes et nombre d'issues fermées.

Modèles et évaluation

1. **Prédiction :** Médiane historique du dépôt.
2. **Baseline :** régression logistique sur TF-IDF et features tabulaires.
3. **Modèle principal :** LightGBM sur features tabulaires et embeddings.
4. **Métriques :** F1-Score, Calibration des probabilités, MAE.

Architecture

1. **Collecte / Streaming :** Apache Kafka
2. **Stockage :** PostgreSQL
3. **Visualisation** : dashboard (volume d'issues, délai, répartition par label).
4. **Serving / Monitoring / Feedback :** API de scoring (Fast API) qui attribue une probabilité de résolution rapide à chaque nouvelle issue.
5. **Évaluation :** évaluation continue sur les issues dont l'issue finale est connue, détection de drift, réapprentissage automatique.

Infrastructure & Déploiement

- Environnement local / Skinner : Exécution et persistance de toute la stack de traitement via Docker
- Microsoft Azure : Hébergement de l'API FastAPI et de l'interface de démo.

Points MLOps forts

- **Labels différés :** la vérité n'est connue qu'à la fermeture, donc l'évaluation en continu se fait sur les issues arrivées à maturité, ce qui justifie naturellement le monitoring.
- **Drift possible** : changement de pratiques d'un projet, nouvelles versions, vocabulaire qui évolue.